

ECS 240: Programming Languages

Lecture 0 – FQ2026

Welcome!

- This is a **graduate course** in **programming languages**
 - Can satisfy core course requirements under the **theory** group
- Current cap: 65 – we have **10 waitlisted** as of this morning

Some will drop – more expected off waitlist; depending on size of waitlist, I can request to increase the cap.

Calendar

Lectures: MWF 1:10-2pm in Olson Hall 158

Final exam: Tue, Dec 8 3:30-5:30pm -- please note that this time is not yet listed in schedulebuilder, but it is the official time for our course slot listed by the registrar.

Join the Piazza! (I will remind you again)

Instructor

Your Instructor: Prof. Stanford (Caleb is also OK)

Started at Davis: July 2023



DavisPL Research Group



TA



TA: Carlos Anguiano

Outline

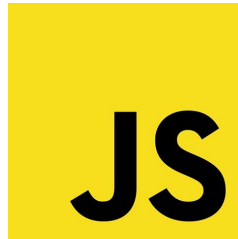
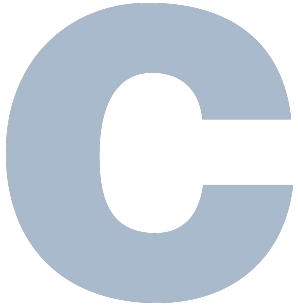
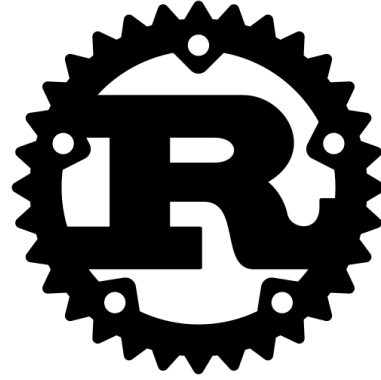
1. What is this class about?
2. Short discussion/activity
3. FAQ, syllabus, and logistics
4. Questions for me
5. Overview of topics (if time)
6. Demos (optional)

Outline

1. What is this class about?
2. Short discussion/activity
3. FAQ, syllabus, and logistics
4. Questions for me
5. Overview of topics (if time)
6. Demos (optional)

What is this class about?

Programming Languages



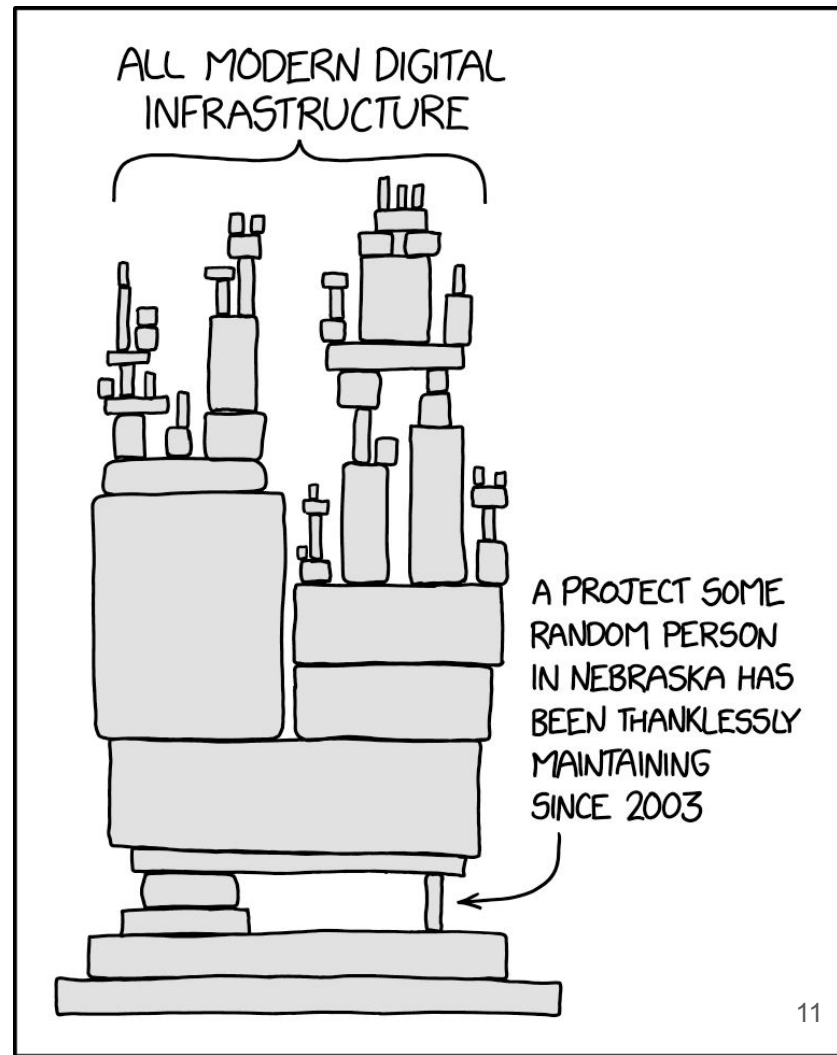
Programming Languages

All software relies on
programming languages!

Programming Languages

All software relies on
programming languages!

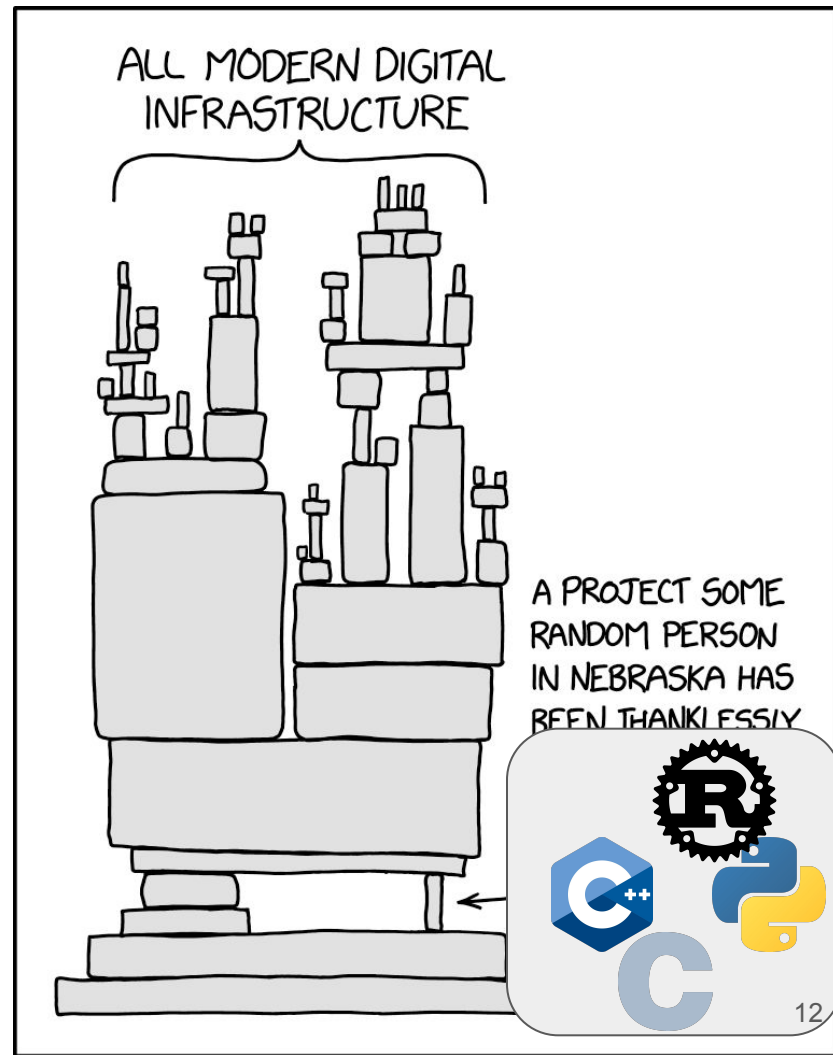
Credit: xkcd #2347



Programming Languages

All software relies on
programming languages!

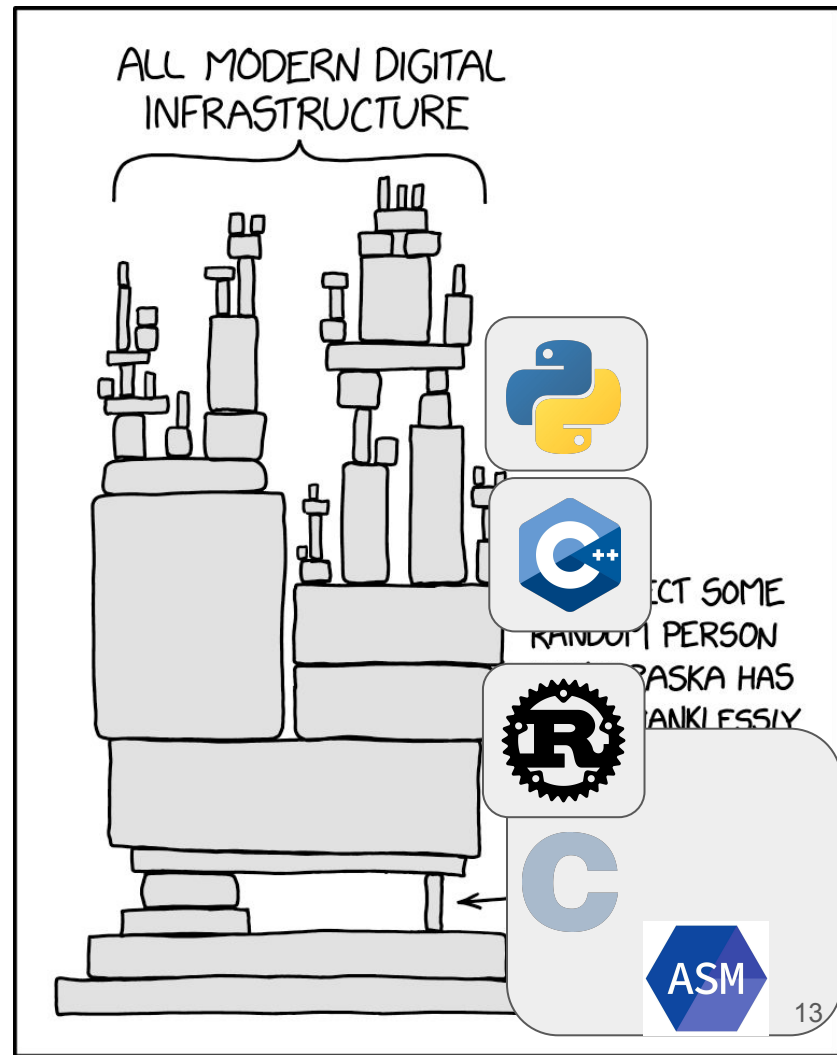
Credit: xkcd #2347



Programming Languages

All software relies on
programming languages!

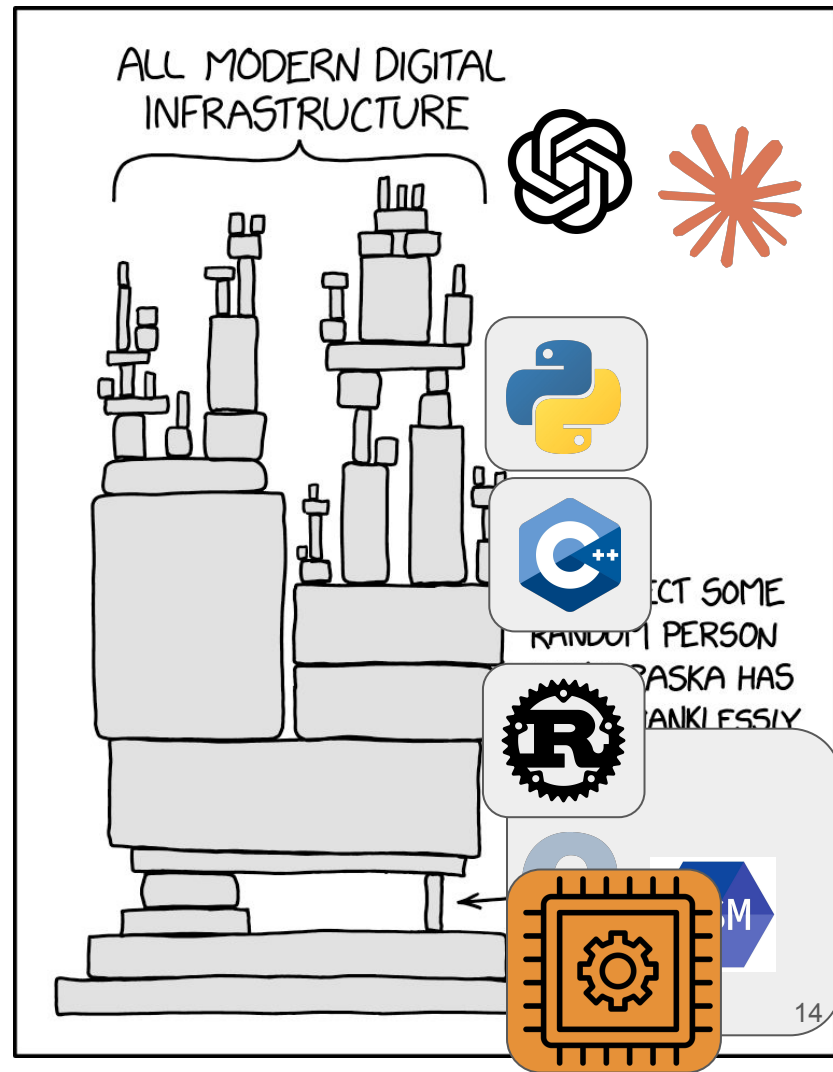
Credit: xkcd #2347



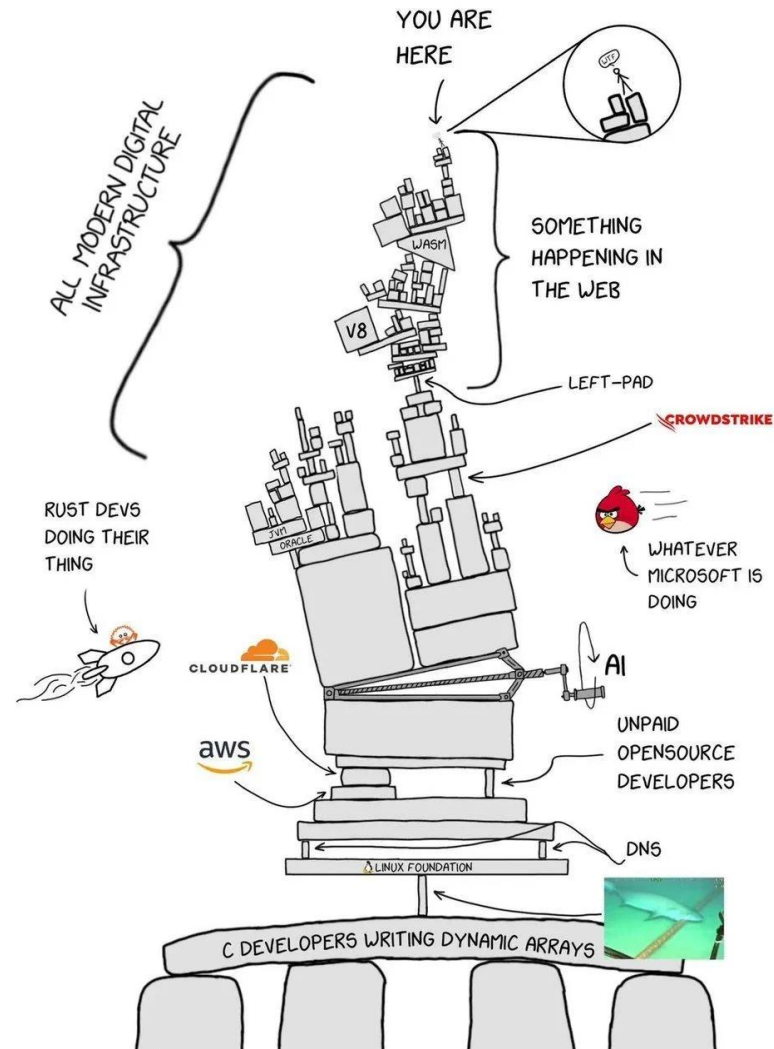
Programming Languages

All software relies on
programming languages!

Credit: xkcd #2347



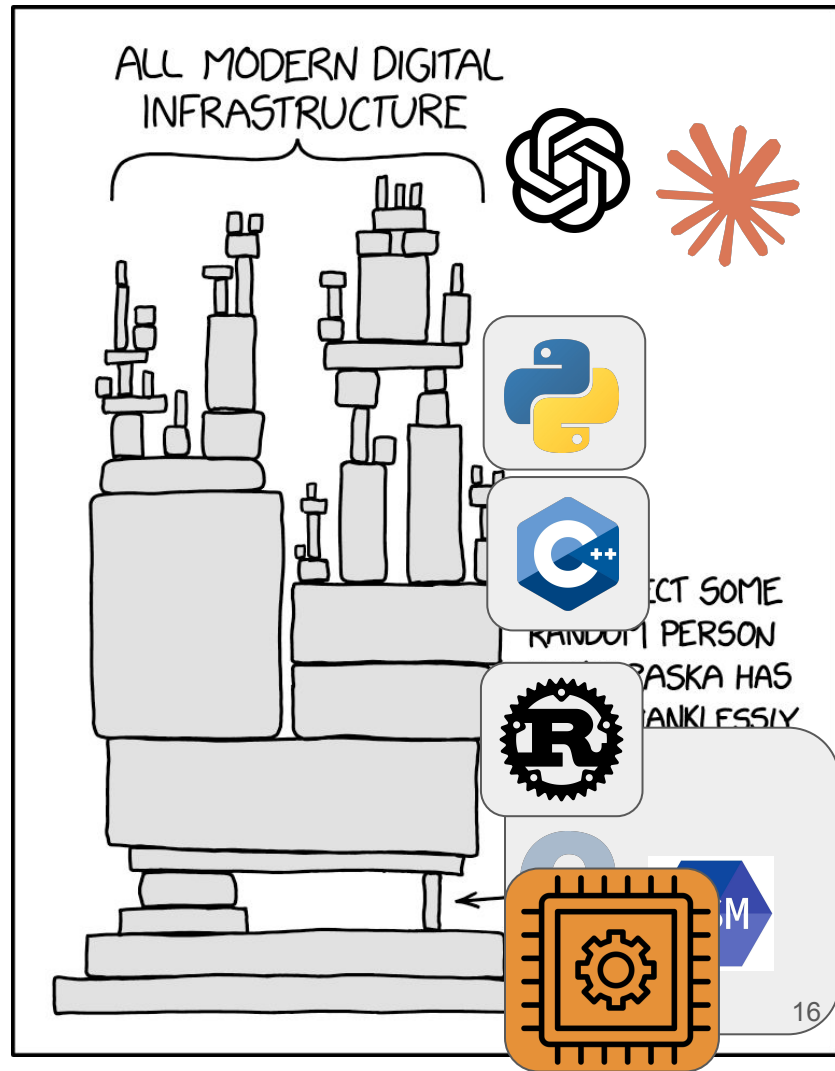
(Alternate comic)



Motivating statement

Errors or **mistakes** in programming languages can cause errors across the software stack

Understanding programming languages is necessary to understand software and how it works.



Motivating examples

Errors or **mistakes**...

Example 1

```
def g(x, y=[]):  
    for i in range(x):  
        y.append(i)  
    print(y)
```



Motivating examples

Errors or **mistakes**...

Example 1

```
def g(x, y=[]):  
    for i in range(x):  
        y.append(i)  
    print(y)
```

Output of g(3)?



Motivating examples

Errors or **mistakes**...

Example 1

```
def g(x, y=[]):  
    for i in range(x):  
        y.append(i)  
    print(y)
```

Output of g(3)? [1, 2, 3]

Output of g(3)?



Motivating examples

Errors or **mistakes**...

Example 1

```
def g(x, y=[]):  
    for i in range(x):  
        y.append(i)  
    print(y)
```

Output of g(3)? [1, 2, 3]

Output of g(3)? [1, 2, 3, 1, 2, 3]



Motivating examples

Errors or **mistakes**...

Example 1

```
def g(x, y=None):  
    if y is None:  
        y = []  
    for i in range(x):  
        y.append(i)  
    print(y)
```





Motivating examples

Errors or **mistakes**...

Example 1

```
def g(x, y=None):  
    if y is None:  
        y = []  
    for i in range(x):  
        y.append(i)  
    print(y)
```

Who's fault is this?

This is a programming language design issue!



Motivating examples

Errors or mistakes...

Example 2

```
void ex1() {  
    vector<int> vec = { 1, 2, 3 };  
  
    int j = 0;  
    for (auto it = vec.begin(); it != vec.end(); ++it) {  
        vec.push_back(j);  
        j++;  
        cout << *it << ", ";  
    }  
}
```



Motivating examples

Errors or mistakes...

Example 2

```
void ex1() {  
    vector<int> vec = { 1, 2, 3 };  
  
    int j = 0;  
    for (auto it = vec.begin(); it != vec.end(); ++it) {  
        vec.push_back(j);  
        j++;  
        cout << *it << ", ";  
    }  
}
```

Vec moves! 

Iterator points to and dumps arbitrary memory



Motivating examples

Errors or mistakes...

Example 2

```
void ex1() {  
    vector<int> vec = { 1, 2, 3 };  
  
    int j = 0;  
    for (auto it = vec.begin(); it != vec.end(); ++it) {  
        vec.push_back(j);  
        j++;  
        cout << *it << ", ";  
    }  
}
```

it expects
temporary
“ownership”
over vec

vec retains
ownership



Memory bug – not possible in “memory safe” languages



Motivating examples



Errors or **mistakes**...

Example 2

THE WHITE HOUSE



FEBRUARY 26, 2024

PRESS RELEASE: Future
Software Should Be
Memory Safe

Memory bug – not possible in “memory safe” languages



Is it a feature... or a bug?

```
def g(x, y=[]):  
    for i in range(x):  
        y.append(i)  
    print(y)
```

```
void ex1() {  
    vector<int> vec = { 1, 2, 3 };  
  
    int j = 0;  
    for (auto it = vec.begin(); it != vec.end(); ++it) {  
        vec.push_back(j);  
        j++;  
        cout << *it << ", ";  
    }  
}
```



The point is not that one language is “better” or “correct”!

- C++ programmers would say that the one on the right is a **feature** – and it's **your problem if you mess it up**

The point is that **programming language design questions affect software – and affect whether or not we build correct programs in it.**

What about AI?



```
def g(x, y=[]):  
    for i in range(x):  
        y.append(i)  
    print(y)
```

```
void ex1() {  
    vector<int> vec = { 1, 2, 3 };  
  
    int j = 0;  
    for (auto it = vec.begin(); it != vec.end(); ++it) {  
        vec.push_back(j),  
        j++;  
        cout << *it << ", ";  
    }  
}
```

AI systems mimic idiomatic C/Python code – they still **have to write code** subject to the **constraints** of that language – and **interface** with code written by humans.

Programming language issues are **even more important** in a world where software is less scrutinized, as they provide constraints on what AI systems can and cannot do, and in exactly what way they misbehave.

Recap: Why Study Programming Languages?

All software relies on programming languages!

Errors or **mistakes** affect the entire the software stack

Programming language design questions are involved in various real-world programming errors

Programming language issues are **even more important** in a world where software is less scrutinized or written by AI.

Recap: Why Study Programming Languages?

Understanding what programming languages will make you a better:

- **Software developer**
- **Compiler, static analysis, type system, or even custom programming language engineer**
- **Hardware developer**
- **Prompt engineer** (probably)

What is a programming language, anyway ?



Let's play a game!

“Is it a programming language”

Let's play a game!



Python

Is it a programming language?

Let's play a game!



Fortran

Is it a programming language?

Let's play a game!



SQL

Is it a programming language?

Let's play a game!



Microsoft Excel

Is it a programming language?

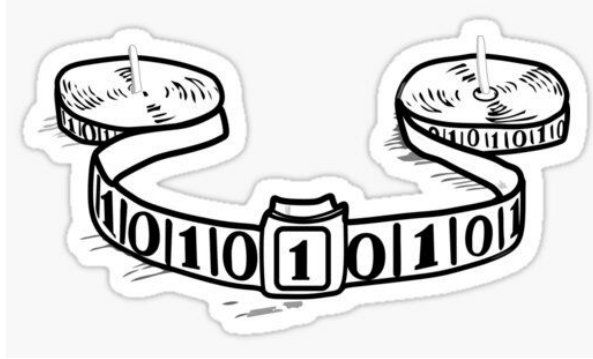
Let's play a game!



ChatGPT

Is it a programming language?

Let's play a game!



Turing Machine

Is it a programming language?

Sharing your answers

forms.gle/8Bupu4ovz1XYaYbAA

(Today's in-class poll – for completion credit only)



(Where we stopped for Sep 23)

ECS 240 – Sep 25

Following along: Piazza -> find “Course Links” pinned in the side bar -> course GitHub -> lecture0 -> [ECS 240 Lecture 0 - FQ26.pdf](#)

Finish slides today; begin lectures/live coding next time

Recap from last time

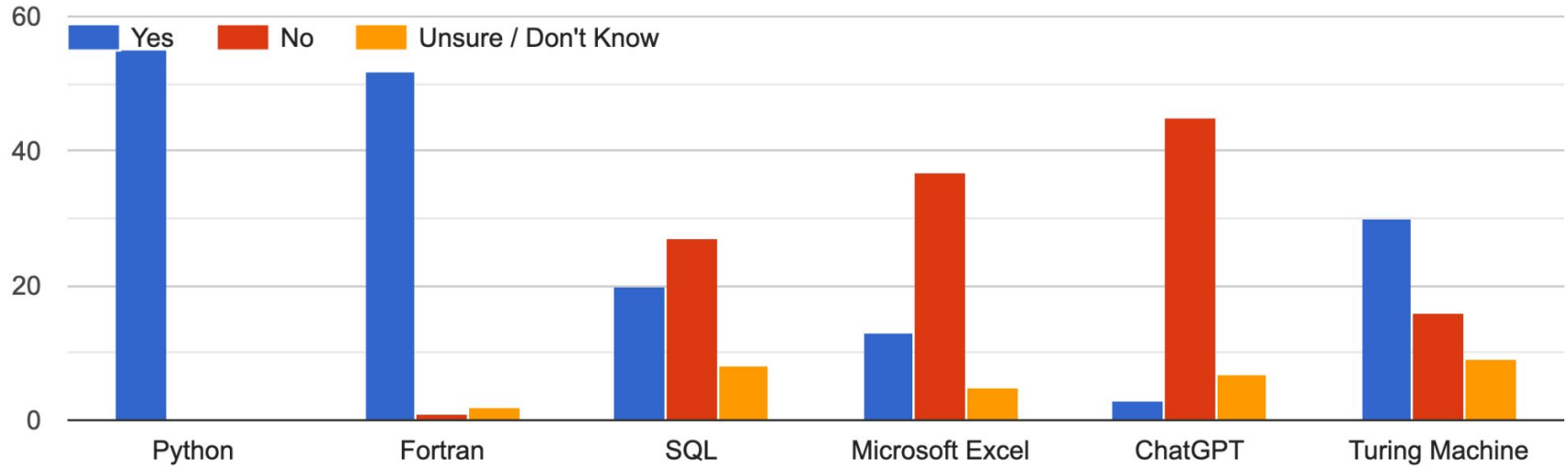
All software relies on programming languages (PL)!

Errors or **mistakes** affect the entire the software stack

Understanding what programming languages will make you a better (software developer, PL tool designer, prompt engineer, etc.)

Results of last time's poll

Is it a programming language?



forms.gle/8Bupu4ovz1XYaYbAA

What is a programming language?

(A little bit of foreshadowing for Lecture 1)

A programming language is any convention that is used for describing instructions for computers to execute.

What is a programming language?

(A little bit of foreshadowing for Lecture 1)

A programming language is any convention that is used for describing instructions for computers to execute.

^^ Very broad!

We get to decide how the convention works, what programs we want to allow/disallow, etc.

What is programming languages (the field)?

The **field of programming languages** includes not only the study of programming languages, but also...

- **Tools** that work with programming languages (Compilers, interpreters, optimizers, static analyzers, type checkers, ...) and their foundations
- Because these tools are about **analyzing** and **understanding** programs! Which requires understanding the language in which that program resides.
 - Without a PL, there is no program

What are the subfields or subareas of programming languages?

PL active research areas

From PLDI 2023:

- Verification (x3)
- PL + ML
- Compilers (x2)
- Concurrency & Parallelism
- Security
- Synthesis
- Probabilistic Programming
- Program Analysis
- Testing
- Types
- Program Logics
- Parsing
- Systems

From POPL 2024:

- Synthesis (x2)
- Types (x5)
- Side effects (x2)
- Verification (x3)
- Regular Expressions & Automata
- Program Logics (x3)
- Concurrency & Parallelism (x2)
- DSLs
- Probabilistic Programming
- Quantum
- Program Analysis
- PL + ML

PL active research areas

Consolidated

1. Logic + Semantics (x8)
2. Compilers + Program Analysis (x4)
3. Verification and Testing (x7)
4. Type Systems (x6)
5. Domain Specific Languages (x6)
6. ML and Synthesis (x5)
7. Concurrency + Parallelism (x3)

PL active research areas

Consolidated

1. Logic + Semantics (x8)
2. Compilers + Program Analysis (x4)
3. Verification and Testing (x7)
4. Type Systems (x6)
5. Domain Specific Languages (x6)
6. ML and Synthesis (x5)
7. Concurrency + Parallelism (x3)

Things I plan to cover

Things we may or may not cover (on request / time permitting)

PL disclaimer

PL is not just about defining “new” programming languages!

PL is also not only about designing languages and compilers

It is about **general tools** used to write, analyze, understand, interpret, compile, optimize, and deploy programs

Plan for class

In the past, ECS 240 has been a focused topics class focused on **static analysis**.

I plan to cover a ✨**survey of various topics**✨ in programming languages, starting with the foundations of programming languages and moving to some of the above.

(Today's poll: preferences on topics to cover)

⚠ Disclaimer ⚠

First time I have taught this class – some things will be **experimental!**



⚠ Disclaimer ⚠

First time I have taught this class – some things will be **experimental!**



Outline

1. What is this class about?
2. Short discussion/activity
3. FAQ, syllabus, and logistics
4. Questions for me
5. Overview of topics (if time)

Outline

1. What is this class about?
2. Short discussion/activity
3. FAQ, syllabus, and logistics
4. Questions for me
5. Overview of topics (if time)

FAQ

Q: Are there any prerequisites?

- Some prior background in mathematical proofs (e.g. **ECS 20 or ECS 120**) is **required**
- Listed prerequisites: some background in programming languages at the undergraduate level (**ECS 140A or ECS 142**) is **helpful, but not required**

Q: Does this course count as a graduation requirement?

A: **Yes**, ECS 240 counts towards your graduation (if you are using the new graduation requirements approved last year)

- **Theory bucket** (3 buckets) – 4 course options

Theory	ECS 220	Theory of Computation	4
	ECS 222A	Design & Analysis of Algorithms	4
	ECS 240	Programming Languages	4
	ECS 256	Probability Models for Computer Science	4

Q: Will I be required to write mathematical proofs?

A: Yes: sometimes!

- This class is in the “theory” core group
- Writing mathematical proofs yourself will be important part of the class, including on some of the HW problems
- Problem sets: LaTeX + Code
- In general, I like to focus on a **hands-on** approach: the tools we will cover can help you check your work

Q: Can I take this class as an undergraduate?

A: Yes, with a PTA, after the add deadline (there is a form you have to fill out, link on department website)

Caveat: you will need to wait 10 days for the drop deadline and make a PTA request; I'll try to get you in, but graduate students may be prioritized for registration, depending on what the department decides.

Q: Will this class be the same as prior iterations of ECS 240?

A: **No, but** it will overlap (part of topics surveyed)

Prior iteration of class was about primarily static analysis.

Q: Will this class overlap with ECS 261 (Program Verification)?

A: **Some** overlap

- I plan to cover some use of formal verification tools for programming language syntax and semantics, and survey some topics in testing and verification

Q: Can you add me off the waitlist / give me a PTA?

Unfortunately I am not allowed to! - all requests have to go through graduate advising

For the waitlist (if applicable) - more on next slide

Waitlist status: **3 waitlisted** as of this morning

Waitlist

During the first/second weeks: Please do attend the class

- Some people will drop
- I'm not allowed to issue any PTAs (department policy)

End of the second week:

- Deadline to drop: 10th day of the quarter/instruction (drop deadline)
- If you have been attending the whole time, I typically request to increase the enrollment cap (but cannot guarantee it will be granted)

TL;DR: Please attend the lectures even if you are on the waitlist.

Q: Should I take this course?

Short answer: **Yes!**

Long answer: **Especially if:**

- You want to learn more about the fundamental principles behind programming languages, their design, theory, use, and application
- You are interested in the foundations of software and what makes programs behave the way they do

Syllabus & logistics

Textbooks (optional) - both **free online**

SOFTWARE FOUNDATIONS

The Software Foundations series is a broad introduction to the mathematical underpinnings of reliable software.

The principal novelty of the series is that every detail is one hundred percent formalized and machine-checked: the entire text of each volume, including the exercises, is literally a "proof script" for the Rocq proof assistant.

The exposition is intended for a broad range of readers, from advanced undergraduates to PhD students and researchers. No specific background in logic or programming languages is assumed, though a degree of mathematical maturity is helpful. A one-semester course can expect to cover *Logical Foundations* plus most of *Programming Language Foundations* or *Verified Functional Algorithms* or *Security Foundations*, or selections from them.

<https://softwarefoundations.cis.upenn.edu/>

Static Program Analysis

Anders Møller and Michael I. Schwartzbach

September 14, 2026

<https://cs.au.dk/~amoeller/spa/spa.pdf>

Graded work

- **Participation (10%)**
- **Homeworks (10%)**
- **Exam (40%)**
- **Final Project (40%)**

Attendance and participation (10%)

Fill out the in-class polls (participation points only)

If you are sick: you may join the class remotely via Zoom (the quality may not be as good)

If you miss class: Lectures are recorded (best-effort).

You can make up the in-class polls at any time



Lectures - live coding

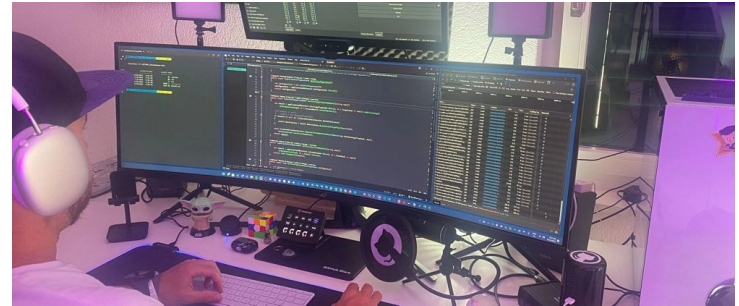
Going forward, most lectures will be by live coding

Lecture notes will be posted before/after the lecture:

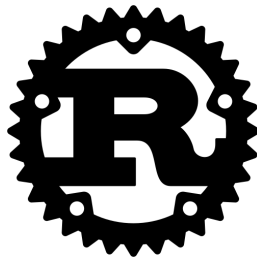
<https://github.com/DavisPL-Teaching/240>

(pinned course links in Piazza)

Bring your laptop!



Lectures - live coding



I plan to work in **Rust** for $\geq$ the first half of the course

- possibly surveying other tools: **Rocq**, **Lean**, **Verus** later on

Why Rust? Benefits of static type system are very helpful for studying languages –



- for example, enum types

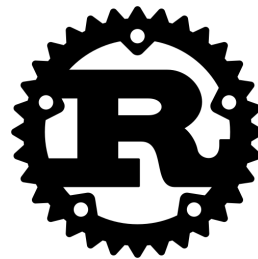
```
(term = Variable | expression)
```



+: Rust can serve as a case study in PL design

HW0 will help you with getting started

Why Rust?



Benefits of static type system are very helpful for studying languages –

- for example, enum types

```
(term = Variable | expression)
```

+: Rust can serve as a case study in PL design

HW0 will help you with getting started

Why proof assistants?

Widely used in PL design and analysis

- Recently in the news due to AI proofs
(Fermat's Last Theorem, Navier-Stokes)
- Renewed interest in **provably safe software**



Help **prove properties** about the language principals we are studying

Help us **check our work**



Homeworks (10%)

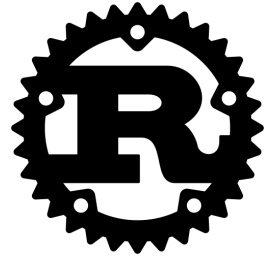
Roughly bi-weekly problem sets
(About 3-4 planned)

Combination of

written problems in LaTeX

and **coding problems**

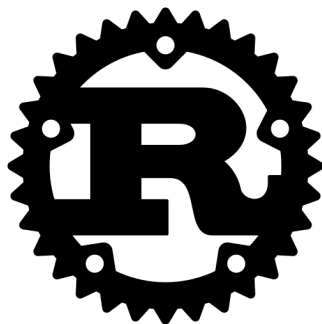
L^AT_EX



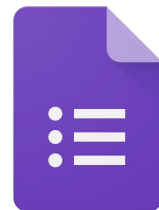
Homework 0: installation help + basics of Rust

(worth 50 points)

(not related to PL topics in particular)



(available early next
week – hopefully
Monday)



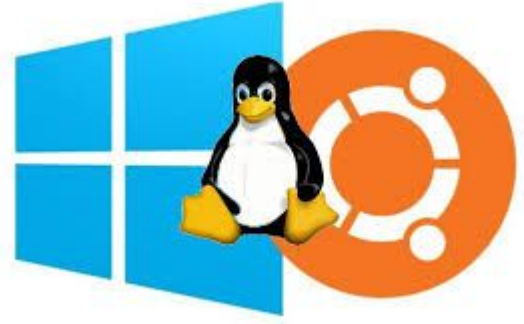
Installation help

I recommend using **MacOS or Linux**

If you are on Windows, I recommend

Windows Subsystem for Linux (WSL)

- Well-engineered tool and very useful to know about when doing software development work in the real world!
- <https://learn.microsoft.com/en-us/windows/wsl/install>



Homework Grading

Most important: please run your code

- Software engineering is about running software!
- Running and testing your code frequently is an important part of developing software in the real world – including ensuring your code works on someone else's machine
-  **Please download and run your code! Code that doesn't run may receive at most half credit**
-  **All HW installation issues should be resolved during HW0**

Exam (40%)

- Final exam
- Please mark your calendar:
 - **Tuesday, December 8 3:30-5:30pm**
- Closed book, single-sided cheat sheet

Project (40%)

Please stay tuned for more details

Likely will choose to build an interpreter, compiler, optimizer, or static analyzer for your own or a custom language

Groups of size 2-4 people (TBD)

- depends on class size – may allow individual projects

I will announce more details in class

Piazza

Please join and monitor Piazza!

[ECS 240 on Piazza](#)

- Don't email me, post to Piazza!
- Make your post public and (if you prefer) anonymous
- Additional platforms: Canvas and Gradescope

AI Policy

I allow the use of AI on HW assignments and the project.

Please be careful! Ensure that your use of AI is helping you learn.

The final exam will be in-class and closed-book.

Collaboration Policy

- **Collaboration is encouraged!**
- Groups of size up to 3 (optional)
- Please list your collaborators at the top of your homework
- Everyone should submit their own solution (for the homeworks); group submissions (for the project)

Schedule note

Please note: there will be **no class on Friday, October 9** as I will be away at a conference.

A Very Rough Schedule (subject to change)

We have 10 weeks, here is a rough plan:

- Week 1-2: introduction, introduction to programming languages concepts; syntax and semantics
- Weeks 3-4: static analysis tools
 - Homework due
- Weeks 5-6: type theory
 - Project proposal: approx. end of week 6
- Weeks 7-8: verification and testing
 - Homework due
- Last 2-3 weeks: final project presentations



(In some order)

Communication

My office hours: Wednesdays 430-530pm, starting **next week**

TA (**Carlos Anguiano**) office hours: TBD (will be posted on Piazza)

Please use Piazza for questions (not email)

Respect and discrimination

Please be nice!

Include everyone in group discussions

Reach out to me if there are any problems

Late policy

Polls and HW0: can be made up

- You don't need to email me about this, just submit the poll! :)

For HW, I cannot guarantee that late work is graded. BUT: I encourage you to keep working on an assignment if you didn't finish it by the deadline. Gradescope will allow you to submit a few days late, and we may choose to grade these late submissions, at our discretion.

- Why no “hard” policy - this is also how it works in the real world :-)
- **Some homeworks may be long - please start early.**

Other disclaimers

- [UC Davis Job Scams Prevention](#)
- [Policies against harassment and discrimination](#)
- [List of resources for student health and well-being](#)
- [GGCS support form](#)
- [SISS](#) for international student issues (and reach out to your graduate advisors)

Outline

1. What is this class about?
2. Short discussion/activity
3. FAQ, syllabus, and logistics
4. Questions for me (and poll)
5. Overview of topics (if time)

Outline

1. What is this class about?
2. Short discussion/activity
3. FAQ, syllabus, and logistics
4. Questions for me (and poll)
5. Overview of topics (if time)

Questions for me?

Poll

Chance for you to give feedback on the topics we cover:

<https://forms.gle/WuoBtJbFc1boHQYB8>

(Please find the form link in the slides - posted in the course GitHub repository)

Outline

1. What is this class about?
2. Short discussion/activity
3. FAQ, syllabus, and logistics
4. Questions for me
5. Overview of topics (if time)

Outline

1. What is this class about?
2. Short discussion/activity
3. FAQ, syllabus, and logistics
4. Questions for me
5. Overview of topics (if time)

Additional slides - Brief overview of topics

*(A *very brief and rough* overview)*

PL active research areas

Consolidated

1. Logic + Semantics (x8)
2. Compilers + Program Analysis (x4)
3. Verification and Testing (x7)
4. Type Systems (x6)
5. Domain Specific Languages (x6)
6. ML and Synthesis (x5)
7. Concurrency + Parallelism (x3)

Things I plan to cover

Things we may or may not cover (on request / time permitting)

PL disclaimer

PL is not just about defining “new” programming languages!

PL is also not only about designing languages and compilers

It is about **general tools** used to write, analyze, understand, interpret, compile, optimize, and deploy programs

1. Foundations of PL: Logic & Semantics

1. Foundations of PL: Logic & Semantics



1. Foundations of PL: Logic & Semantics

Every programming language is defined by its **syntax** and its **semantics**

1. Foundations of PL: Logic & Semantics

Syntax:

```
Variable ::= x | y | z  
Term ::= 0 | 1 | Term + Term | Variable  
Program ::= set Variable = Term | return Term | Program; Program
```

1. Foundations of PL: Logic & Semantics

Syntax:

Variable ::= x | y | z

Term ::= 0 | 1 | Term + Term | Variable

Program ::= set Variable = Term | return Term | Program; Program

1. Foundations of PL: Logic & Semantics

Syntax:

```
Variable ::= x | y | z  
Term ::= 0 | 1 | Term + Term | Variable  
Program ::= set Variable = Term | return Term | Program; Program
```

- What does “x” mean?
- What does 1 mean?
- $1 + 1 = 3$?

1. Foundations of PL: Logic & Semantics

Syntax:

```
Variable ::= x | y | z  
Term ::= 0 | 1 | Term + Term | Variable  
Program ::= set Variable = Term | return Term | Program; Program
```

- What does “x” mean?
- What does 1 mean?
- $1 + 1 = 3?$

Sure! :-)

1. Foundations of PL: Logic & Semantics

Semantics:

```
Variable ::= x | y | z  
Term ::= 0 | 1 | Term + Term | Variable  
Program ::= set Variable = Term | return Term | Program; Program
```

“P evaluates to Q”

“P on input X evaluates to Q

returning Y in context C”

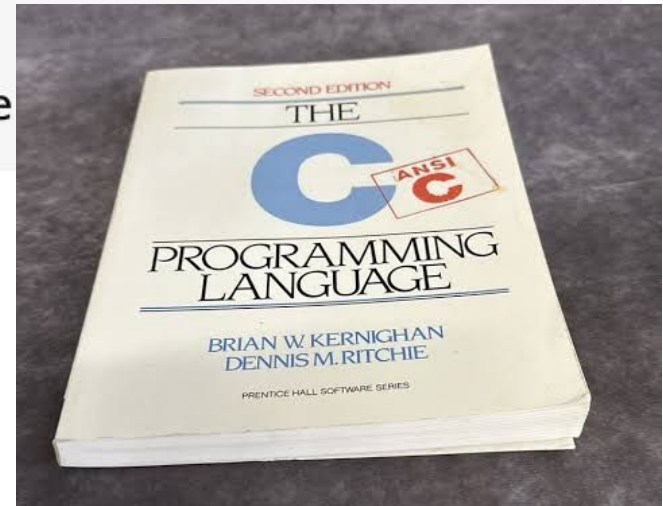
1. Foundations of PL: Logic & Semantics

Semantics:

```
Variable ::= x | y | z  
Term ::= 0 | 1 | Term + Term | Variable  
Program ::= set Variable = Term | return Te
```

“P evaluates to Q”

**“P on input X evaluates to Q
returning Y in context C”**

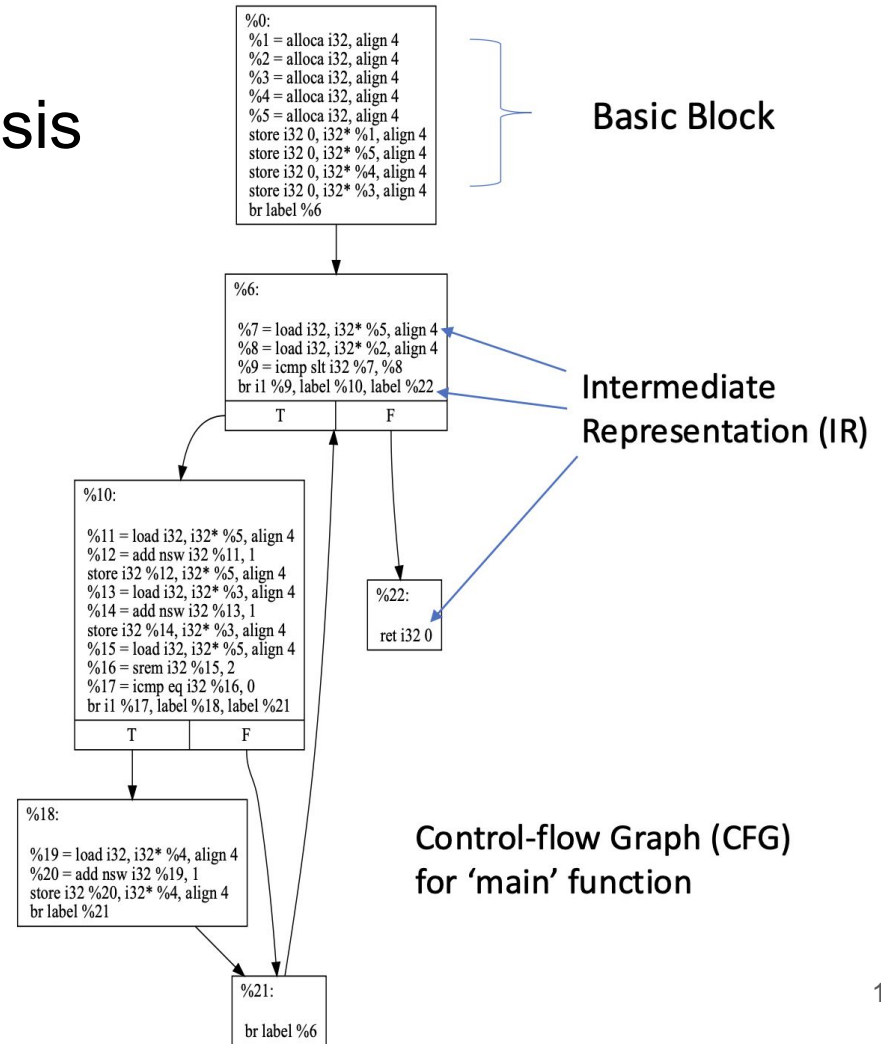


2. Compilers and Static Analysis

```
int main() {  
    int n, x, y, i;  
    x = y = i = 0;  
    while (i < n) {  
        i++; x++;  
        if (i%2 == 0) y++; }  
    return 0;  
}
```

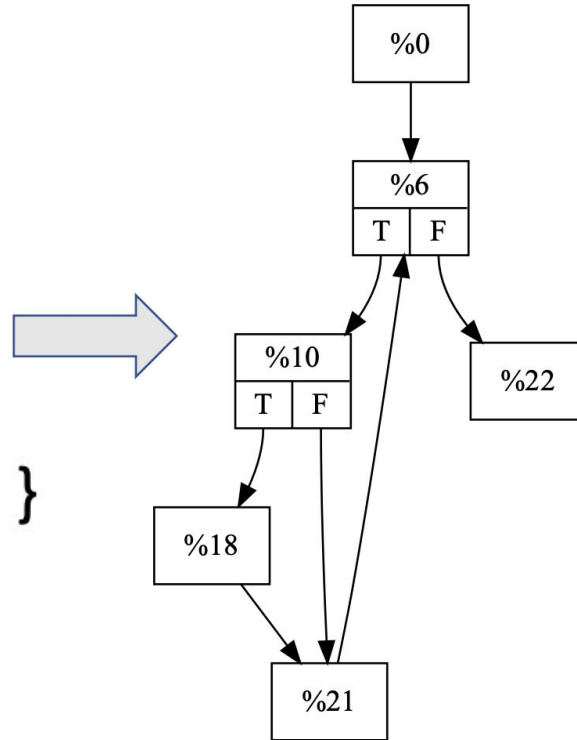
2. Compilers and Static Analysis

```
int main() {  
    int n, x, y, i;  
    x = y = i = 0;  
    while (i < n) {  
        i++; x++;  
        if (i%2 == 0) y++;  
    }  
    return 0;  
}
```

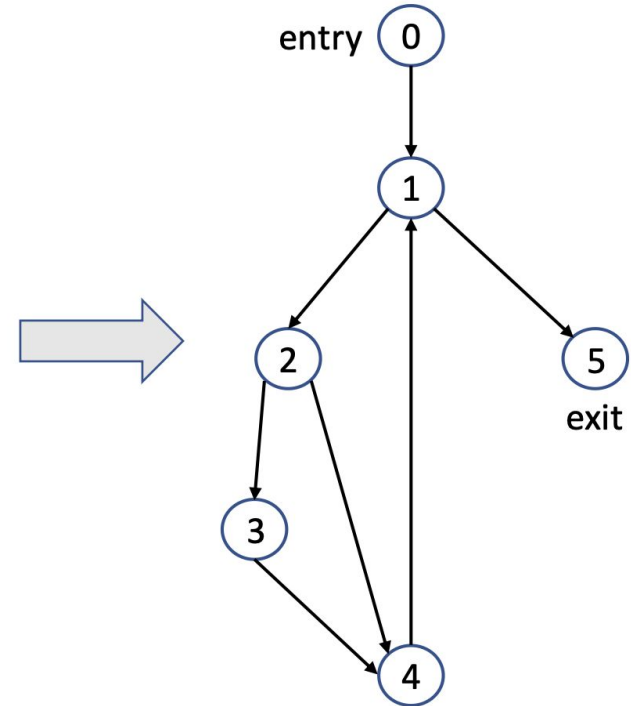


2. Compilers and Static Analysis

```
int main() {  
    int n, x, y, i;  
    x = y = i = 0;  
    while (i < n) {  
        i++; x++;  
        if (i%2 == 0) y++;  
    }  
    return 0;  
}
```



CFG for 'main' function

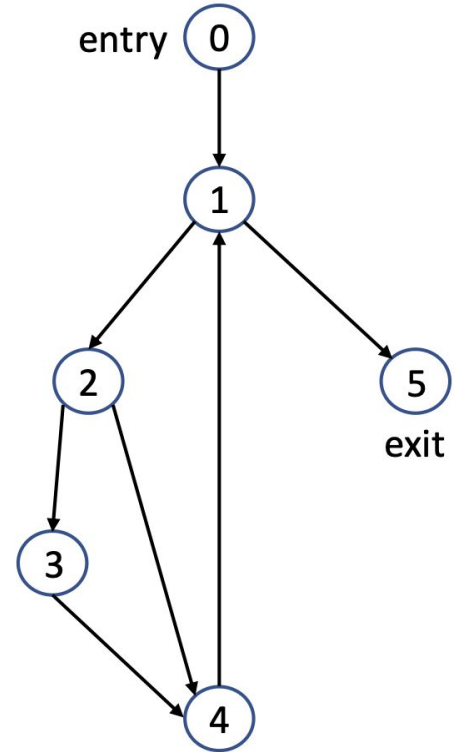


2. Compilers and Static Analysis

```
int main() {  
    int n, x, y, i;  
    x = y = i = 0;  
    while (i < n) {  
        i++; x++;  
        if (i%2 == 0) y++;  
    }  
    return 0;  
}
```

Can be used to:

- Compile to a binary
(Compiler)
- Check for program properties (does the program crash?
Does it infinite loop?)
(Program analysis)

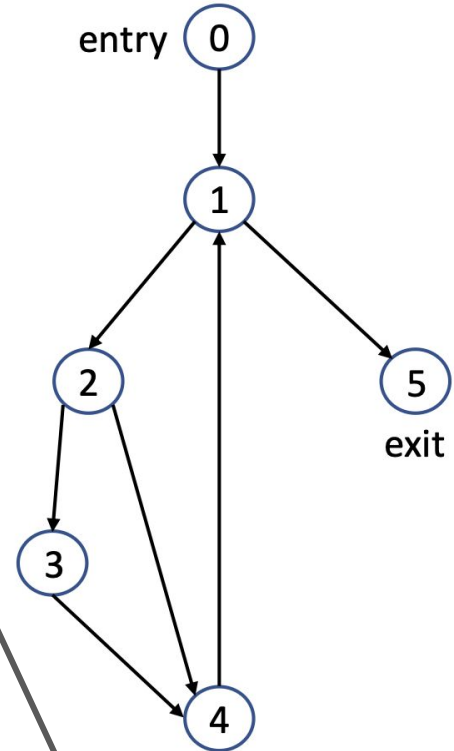


2. Compilers and Static Analysis

```
int main() {  
    int n, x, y, i;  
    x = y = i = 0;  
    while (i < n) {  
        i++; x++;  
        if (i%2 == 0) y++;  
    }  
    return 0;  
}
```

Can be used to:

- Compile to a binary
(Compiler)
- Check for program properties (does the program crash?
Does it infinite loop?)
(Program analysis)



Closely related!

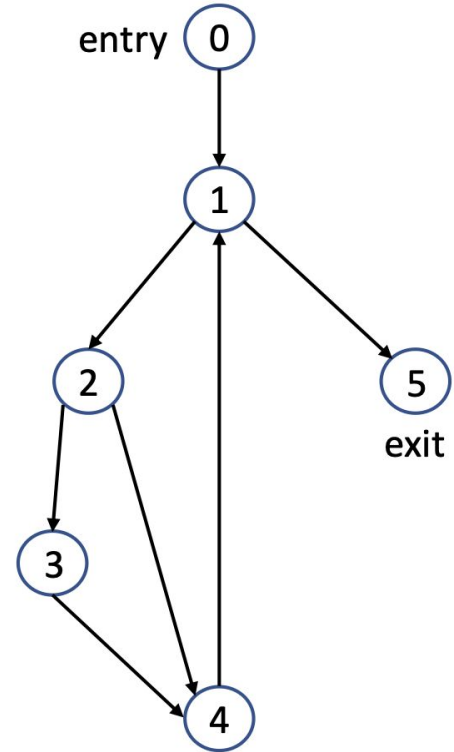
2. Compilers and Static Analysis

Sound abstraction:

Program analysis is undecidable

Example:

- If control can flow from one basic block to another, then there has to be path between the corresponding nodes in the CFG
- But the converse may not be true!



3. Verification and Testing

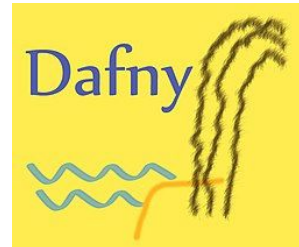
So you've written a program in a programming language ...
how do you know it's correct?

3. Verification and Testing



1. Write down what the program **does (and should do)**
2. Come up with a **rigorous mathematical** argument
3. Use **automatic tools** check (verify) that argument

Z3



LEVNTM

3. Verification and Testing



1. Write down what the program **does (and should do)**
2. Come up with a **rigorous mathematical** argument
3. Use **automatic tools** check (verify) that argument



3. Verification and Testing



1. Write down what the program **does (and should do)**
2. Come up with a **rigorous mathematical** argument
3. Use **automatic tools** check (verify) that argument

Z3

Dafny

$\exists$ M N TM

ECS 189C and ECS 261

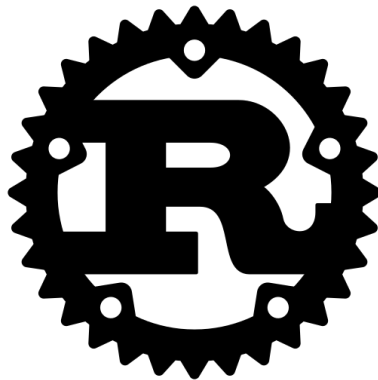
Lots of interest nowadays (due to AI)

4. Type Systems

Type systems can be used to

Formally prove that your code always
satisfies some “**safety checks**”

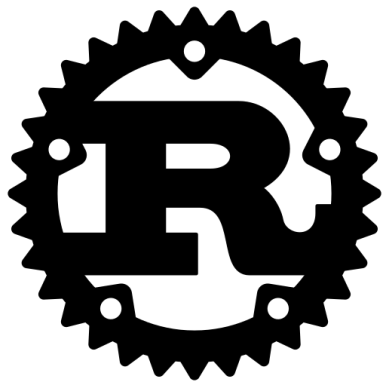
What safety checks? Up to you!



4. Type Systems

Type systems can be used to

Formally prove that your code always
satisfies some “**safety checks**”



What safety checks? Up to you!

I like to think of type systems as a way for you to **constrain the programmer** to ensure they write the code in the way you want

4. Type Systems - example

Understanding Real-World Concurrency Bugs in Go

Tengfei Tu*

BUPT, Pennsylvania State University
tutengfei.kevin@bupt.edu.cn

Linhai Song

Pennsylvania State University
songlh@ist.psu.edu

Xiaoyu Liu

Purdue University
liu1962@purdue.edu

Yiying Zhang

Purdue University
yiying@purdue.edu

```
1      for i := 17; i <= 21; i++ { // write
2 -      go func() { /* Create a new goroutine */
3 +      go func(i int) {
4          apiVersion := fmt.Sprintf("v1.%d", i) // read
5          ...
6 -      }()
7 +      }(i)
8      }
```

Figure 8. A data race caused by anonymous function.

4. Type Systems - example

Understanding Real-World Concurrency Bugs in Go

Tengfei Tu*
BUPT, Pennsylvania State University
tutengfei.kevin@bupt.edu.cn

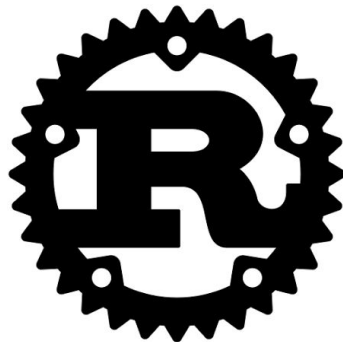
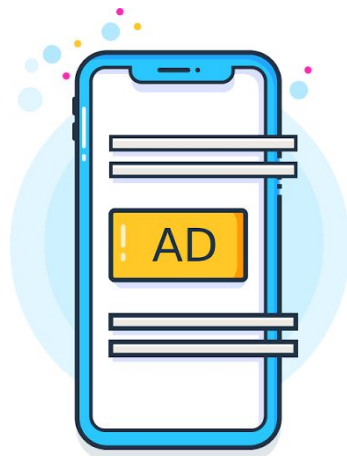
Linhai Song
Pennsylvania State University
songlh@ist.psu.edu

Xiaoyu Liu
Purdue University
liu1962@purdue.edu

Yiying Zhang
Purdue University
yiying@purdue.edu

```
1   for i := 17; i <= 21; i++ { // write
2 -   go func() { /* Create a new goroutine */
3 +   go func(i int) {
4       apiVersion := fmt.Sprintf("v1.%d", i) // read
5       ...
6 -   }()
7 +   }(i)
8   }
```

Figure 8. A data race caused by anonymous function.



4. Type Systems - quotes

Luca Cardelli (attributed):

“The purpose of a type system is to rule out bad programs.”



i.e.:

“A type system is defined by the programs it rejects.”

4. Type Systems - quotes

Robin Milner (paraphrased):

"well-typed programs cannot go wrong"



A Theory of Type Polymorphism in Programming

ROBIN MILNER

Computer Science Department, University of Edinburgh, Edinburgh, Scotland

Received October 10, 1977; revised April 19, 1978

4. Type Systems - quotes

Robin Milner (paraphrased):

"well-typed programs cannot go wrong"

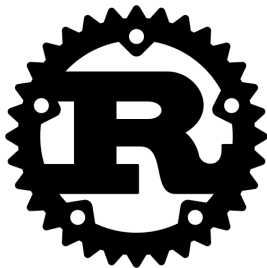


A Theory of Type Polymorphism in Programming

ROBIN MILNER

Computer Science Department, University of Edinburgh, Edinburgh, Scotland

Received October 10, 1977; revised April 19, 1978



That's it!

Reminder to fill out the poll:

<https://forms.gle/WuoBtJbFc1boHQYB8>

We'll talk more and get into **specific tools** next time.

Programming languages (PL) and related fields

Computer Science

